

Behavioral Cloning

Writeup

Behavioral Cloning Project

The goals / steps of this project are the following:

- Use the simulator to collect data of good driving behavior
- Build, a convolution neural network in Keras that predicts steering angles from images
- Train and validate the model with a training and validation set
- Test that the model successfully drives around track one without leaving the road
- Summarize the results with a written report

Rubric Points

Here I will consider the [rubric points](#) individually and describe how I addressed each point in my implementation. This writeup follows the template.

Files Submitted & Code Quality

1. Submission includes all required files and can be used to run the simulator in autonomous mode

My project includes the following files:

- model.py containing the script to create and train the model
 - drive.py for driving the car in autonomous mode
 - model.h5 containing a trained convolution neural network
 - writeup_report.pdf summarizing the results
-
- The mp4 file generated with run1

2. Submission includes functional code

Using the Udacity provided simulator and my drive.py file, the car can be driven autonomously around the track by executing

```
python drive.py model.h5
```

3. Submission code is usable and readable

The model.py file contains the code for training and saving the convolution neural network. The file shows the pipeline I used for training and validating the model, and it contains comments to explain how the code works.

Model Architecture and Training Strategy

1. An appropriate model architecture has been employed

As suggested in the instructions, I followed the NVidia pipeline architecture. The paper describing it is available at <https://arxiv.org/pdf/1604.07316v1.pdf>

My model consists of:

A cropping layer that remove the top and bottom part of the image

```
model.add(Cropping2D(cropping=((50,20), (0,0)), input_shape=(160,320,3)))
```

A Keras lambda layer to normalize the data

```
model.add(Lambda (lambda x: (x / 255.0) - 0.5) )
```

3 convolutional layers with 5x5 kernel and 2x2 stride each followed by an Exponential Linear Unit activation function.

ELUs alleviate the vanishing gradient problem via the identity for positive values as explained in this paper *Fast and Accurate Deep Network Learning by Exponential Linear Units (ELUs)*

available at <https://arxiv.org/abs/1511.07289>

```
model.add(Conv2D(24, (5, 5), strides=(2, 2), activation="elu"))
model.add(Conv2D(36, (5,5), strides=(2, 2), activation="elu"))
model.add(Conv2D(48, (5,5), strides=(2, 2), activation="elu"))
```

2 convolutional layers with 3x3 kernel and 1x1 stride each followed by an Exponential Linear Unit activation function.

```
model.add(Conv2D(64, (3,3), activation="elu"))  
model.add(Conv2D(64, (3,3), activation="elu"))
```

A flattening layer to reduce from 2d.

```
model.add(Flatten())
```

A fully connected layer followed by Exponential Linear Unit activation function

```
model.add(Dense(100, activation="elu"))
```

A dropout layer to prevent overfitting

```
model.add(Dropout(0.25))
```

3 fully connected layers followed by Exponential Linear Unit activation function

```
model.add(Dense(50, activation="elu"))  
model.add(Dense(10, activation="elu"))  
model.add(Dense(1))
```

2. Attempts to reduce overfitting in the model

The model contains a dropout layer after the first fully connected layer in order to reduce overfitting.

The model was trained and validated on datasets to ensure that the model was not overfitting. 20% of the dataset was kept for validation.

The model was tested by running it through the simulator and ensuring that the vehicle could stay on the track.

3. Model parameter tuning

The model used an adam optimizer, so the learning rate was not tuned manually

```
model.compile(loss='mse',optimizer='adam')
```

4. Appropriate training data

Training data was chosen to keep the vehicle driving on the road.

I used the data provided by Udacity.

For details about the training data, see the next section.

Model Architecture and Training Strategy

1. Solution Design Approach

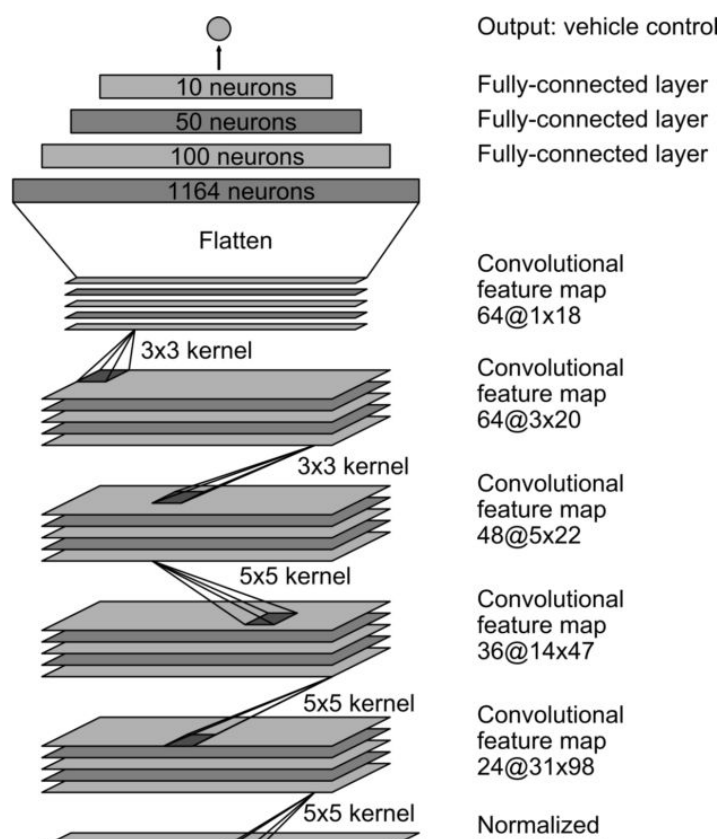
I made sure to make good use of all the resources provided to me by Udacity in order not to waste my GPU workspace time.

The video instructions were providing many suggestions, and the Cheatsheet was indicating clear steps to implement this project as well as the pitfalls to avoid.

The Cheatsheet is available here:

<https://s3-us-west-1.amazonaws.com/udacity-selfdrivingcar/Behavioral+Cloning+Cheatsheet+-+CarND.pdf>

The overall strategy for deriving a model architecture was to follow the suggestion of using the NVidia model and to slightly modify it to better match the project.



NVidia model as presented in
[End-to-End Deep Learning for Self-Driving Cars](#)

By using a model that was already state of the art, I could quickly assess its performance and play around the layers much faster. To check each modification I ran it on a few images, and could see how it was doing. This was advised in the cheatsheet.

In order to gauge how well the model was working, I split my image and steering angle data into a training and validation set. I found that my first attempts had a low mean squared error on the training set but a high mean squared error on the validation set. This implied that the model was overfitting. I reduced the number of epochs to only 2.

To combat the overfitting, I modified the model with a dropout layer after the first fully connected layer.

At the end of the process, the vehicle is able to drive autonomously around the track without leaving the road.

2. Final Model Architecture

The final model architecture consists of the following

Input 160x320x3 RGB image	
Cropping	<code>model.add(Cropping2D(cropping=((50,20), (0,0)), input_shape=(160,320,3)))</code>
Normalization	<code>model.add(Lambda (lambda x: (x / 255.0) - 0.5))</code>
Convolutional layer dimensionality of the output space 24, 5×5 kernel , 2x2 stride, Exponential Linear Unit activation function	<code>model.add(Conv2D(24, (5, 5), strides=(2, 2), activation="elu"))</code>
Convolutional layer dimensionality of the output space 36, 5×5 kernel , 2x2 stride, Exponential Linear Unit activation function	<code>model.add(Conv2D(36, (5,5), strides=(2, 2), activation="elu"))</code>
Convolutional layer dimensionality of the output space 48, 5×5 kernel , 2x2 stride, Exponential Linear Unit activation function	<code>model.add(Conv2D(48, (5,5), strides=(2, 2), activation="elu"))</code>
Convolutional layer dimensionality of the output space 64, 3x3 kernel, 1x1 stride, Exponential Linear Unit activation function	<code>model.add(Conv2D(64, (3,3), activation="elu"))</code>

Convolutional layer dimensionality of the output space 64, 3x3 kernel, 1x1 stride, Exponential Linear Unit activation function	<code>model.add(Conv2D(64, (3,3), activation="elu"))</code>
Flattening	<code>model.add(Flatten())</code> Keras does automatic shape inference
Fully connected layer, Exponential Linear Unit activation function	<code>model.add(Dense(100, activation="elu"))</code>
Dropout	<code>model.add(Dropout(0.25))</code>
Fully connected layer, Exponential Linear Unit activation function	<code>model.add(Dense(50, activation="elu"))</code>
Fully connected layer, Exponential Linear Unit activation function	<code>model.add(Dense(10, activation="elu"))</code>
Fully connected layer	<code>model.add(Dense(1))</code>

3. Creation of the Training Set & Training Process

To capture good driving behavior, I used the data provided by Udacity in the directory above the home directory and which is only accessible in GPU enabled mode.

Here is an example left, central, and right camera images from the original data.



I performed augmentation of the data set of the center, left, and right camera by flipping images just as suggested in the instructions.

As suggested in the Cheatsheet and in the NVidia paper, correction was applied to the left and right camera images.

```
correction = 0.2
```

I used a generator so that it uses less memory, as suggested in the instructions.

I finally randomly shuffled the data set and put 20% of the data into a validation set.

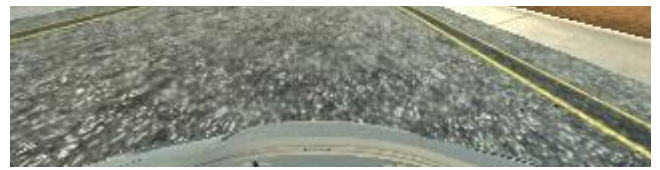
I used this training data for training the model. The validation set helped determine if the model was over or under fitting. The ideal number of epochs was 2. More epochs were leading to overfitting.

I used an adam optimizer so that manually training the learning rate wasn't necessary.

As explain above, I cropped the images to avoid extraneous data that were not useful to learn the road.



Before cropping



Cropped image